

# 模块四：评测任务管理与执行 (Evaluation Task & Execution)。

## 接口说明

该接口用于评测任务的全生命周期管理。包括创建评测配置、触发执行实例（Execution）、实时追踪 Agent 推理轨迹（Trace）以及查看最终标注结果。

### 1. 评测任务 (Eval Task) 关键字段说明

| 参数名               | 类型          | 是否必填 | 说明                                                                         |
|-------------------|-------------|------|----------------------------------------------------------------------------|
| agent_version_id  | String      | 是    | 关联被测 Agent 版本的 ID                                                          |
| dataset_id        | Long/String | 是    | 关联评测数据集 ID                                                                 |
| evaluation_config | JSON        | 是    | 配置本次任务选用的指标及其权重，如<br><pre>{"metrics": [1, 2], "weights": {"1": 0.6}}</pre> |
| status            | String      | 系统维护 | 任务状态： PENDING ，<br>RUNNING ，<br>COMPLETED ，<br>FAILED ， STOPPED            |
| result_summary    | JSON        | 系统维护 | 任务结束后的聚合得分快照                                                               |

### 2. 执行批次 (Execution) 关键字段说明

| 参数名                 | 类型          | 是否必填 | 说明      |
|---------------------|-------------|------|---------|
| task_id             | Long/String | 是    | 所属任务 ID |
| performance_metrics | JSON        | 系统维护 |         |

|             |         |      |                                   |
|-------------|---------|------|-----------------------------------|
|             |         |      | 统计信息：平均延迟、总Token 消耗等              |
| total_cases | Integer | 系统维护 | 数据集总样本数                           |
| completed   | Integer | 系统维护 | 已完成样本数，用于计算 <code>progress</code> |

### 3. 完整table

#### eval\_tasks（评测任务主表）

- 目标：承载“谁在什么数据上按什么配置执行评估”。
- 字段原因：
  - id/name/description：任务标识和可读性。
  - agent\_version\_id（外键，RESTRICT）：禁止删掉正在被任务引用的版本，防止历史任务失效。
  - dataset\_id（外键，RESTRICT）：同理，保证任务可回放。
  - evaluation\_config（JSON）：评估方法组合、权重、维度可能频繁变化。
  - status：任务状态机核心（pending/running/completed/failed/stopped）。
  - progress：进度百分比；加了 0 到 1 的检查约束，避免脏值。
  - total\_claims：针对事实核查场景的中间统计（来自 plan 事件）。
  - result\_summary（JSON）：任务结束后的聚合结果，读接口更快。
  - metadata：任务级扩展参数。
  - created\_at/updated\_at/started\_at/completed\_at：全生命周期审计。
- 索引原因：
  - status + created\_at：任务列表常见筛选与排序路径。

---

#### eval\_trace\_logs（过程追踪日志）

- 目标：保存执行过程中的 thinking/tool\_call/error，支持可解释性和排障。
- 字段原因：
  - id（自增）：日志高频写入，整数主键更高效。
  - task\_id（外键）：绑定任务。
  - claim\_id（可空）：允许“全局错误”这类非声明级事件。
  - event\_type：快速过滤事件类型。

- content (JSON)：完整保留原始事件，避免信息丢失。
  - timestamp：时间序追踪。
  - 索引原因：
    - task\_id + timestamp：按任务时间线读取是最常见查询。
- 

### **eval\_annotations (最终标注结果)**

- 目标：保存每条声明的最终判断，供统计与结果展示。
  - 字段原因：
    - id (自增)：写多读多，简单稳定。
    - task\_id (外键)：归属任务。
    - claim\_id：连接声明级语义。
    - text/start\_offset/end\_offset：定位原文片段，支持前端高亮。
    - error\_type：分类标签（事实错误、逻辑谬误等）。
    - confidence：模型置信度。
    - reasoning：判定解释文本。
    - evidence\_urls (JSON)：证据链接数组。
    - timestamp：事件时间。
  - 索引原因：
    - task\_id + claim\_id：任务详情页按声明回查高频路径。
- 

### **executions (执行批次)**

- 目标：把“任务”与“一次具体运行”区分开，支持重跑和运行态统计。
- 字段原因：
  - id/task\_id (外键)：执行实例归属任务。
  - status/progress：运行态管理。
  - total\_cases/completed/pending/passed/failed/skipped：执行计数快照，避免临时聚合成本。
  - performance\_metrics (JSON)：性能指标扩展位（延迟、token等）。
  - errors (JSON)：执行级错误集合。
  - duration/started\_at/completed\_at：耗时分析基础。
- 约束与索引：

- progress 检查约束（0 到 1）。
- status + started\_at 索引，便于看“最近运行中/失败执行”。

### execution\_cases（单用例执行明细）

- 目标：落地每个 case 的输入、输出、步骤与评分，支持深度诊断。
- 字段原因：
  - id（自增）+ execution\_id（外键）+ case\_id：定位到具体执行中的具体样本。
  - status：每条样本执行状态。
  - input\_data/output\_data/expected\_output（JSON）：完整保真存储。
  - execution\_details（JSON）：步骤、工具调用、token、response\_time。
  - evaluation\_results（JSON）：多评估方法结果可并存。
  - error：失败原因文本。
  - started\_at/completed\_at：单样本性能分析。
- 设计意义：
  - 这是“结果可解释性”和“失败复盘”的关键表。

## 基础配置

1. 响应格式：统一返回 `Result<VO>` 对象。
2. 异步处理：创建执行 (Run) 是耗时操作，后端立即返回 `execution_id`，过程数据通过日志接口轮询。
3. Host: `localhost:8080`
4. 路径前缀: `/api/eval`

## 接口列表

| 接口名称 | 接口描述 | Http方法 | URL |
|------|------|--------|-----|
|------|------|--------|-----|

|                    |                              |      |                                       |
|--------------------|------------------------------|------|---------------------------------------|
| createTask         | 创建评测任务配置                     | POST | /api/eval/tasks                       |
| getTaskList        | 分页查询任务列表                     | GET  | /api/eval/tasks                       |
| startExecution     | 启动评测运行                       | POST | /api/eval/tasks/{id}/run              |
| getExecutionStatus | 获取当前运行进度与计数                  | GET  | /api/eval/executions/{id}/status      |
| getTraceLogs       | 获取推理过程轨迹<br>(Thinking/Tools) | GET  | /api/eval/executions/{id}/traces      |
| getAnnotations     | 获取最终核查标注结果                   | GET  | /api/eval/executions/{id}/annotations |
| getCaseDetails     | 获取单个样本的深度诊断数据                | GET  | /api/eval/cases/{caseId}              |

## 关键接口请求示例

### 1. 创建评测任务 (createTask)

POST localhost:8080/api/eval/tasks

#### Request Body:

代码块

```
1  {
2      "name": "Sonnet-V1 事实核查准确性评测",
3      "agent_version_id": "550e8400-e29b-41d4-a716-446655440000",
4      "dataset_id": 1,
5      "evaluation_config": {
6          "selected_metrics": [1, 5, 8],
7          "params": { "ragas_llm": "gpt-4o" }
8      },
9      "description": "对比 ReAct 模式在科技新闻下的表现"
10 }
```

## 2. 启动评测运行 (startExecution)

**POST** `localhost:8080/api/eval/tasks/101/run`

**说明：**每次点击“开始评测”，都会生成一个新的 `execution_id`。

**Response:**

代码块

```
1  {
2    "code": 200,
3    "data": {
4      "execution_id": 20240408001,
5      "status": "RUNNING",
6      "started_at": "2024-04-08T15:00:00"
7    },
8    "msg": "评测已在后台启动"
9  }
```

## 3. 获取推理过程轨迹 (getTraceLogs)

**GET** `localhost:8080/api/eval/executions/20240408001/traces?claim_id=c001`

**说明：**对应需求中的“面向过程评估”，前端根据此数据画出 Agent 的思考链路。

**Response:**

代码块

```
1  {
2    "code": 200,
3    "data": [
4      {
5        "event_type": "thinking",
6        "content": { "thought": "正在搜索 2023 年 GDP 数据..." },
7        "timestamp": "2024-04-08T15:01:00"
8      },
9      {
10       "event_type": "tool_call",
11       "content": { "tool": "google_search", "query": "China GDP 2023",
12         "output": "5.2%" },

```

```
12         "timestamp": "2024-04-08T15:01:05"
13     }
14 ]
15 }
```

## 4. 获取最终核查标注 (getAnnotations)

GET `localhost:8080/api/eval/executions/20240408001/annotations`

**说明：**对应需求中的“面向结果评估”，用于前端高亮显示原文中的错误。

**Response:**

代码块

```
1  {
2      "code": 200,
3      "data": [
4          {
5              "claim_id": "c001",
6              "text": "中国GDP增速为8.5%",
7              "error_type": "factual_error",
8              "reasoning": "世界银行数据显示为5.2%",
9              "confidence": 0.95,
10             "evidence_urls": ["https://worldbank.org/..."]
11         }
12     ]
13 }
```

## 业务逻辑解释

### 1. “任务(Task)”与“执行(Execution)”的区分原因

- **任务 (Task)** 是静态配置：它定义了“用 A 测 B”。
- **执行 (Execution)** 是动态实例：同一个任务可以跑多次（比如 Agent 代码改了，在同一个数据集上重跑）。

- **价值：**支持**纵向对比**。你可以对比同一个 Task 下“昨天跑的”和“今天跑的”分数差异。

## 2. 状态机的流转

1. 用户点击 `Run` -> 创建 `Execution` (Status: `PENDING`)。
2. 后端 Worker 捡起任务 -> 调用 Agent 接口 -> (Status: `RUNNING`)。
3. 后端实时解析 Agent 的 SSE 事件：
  - 解析到 `thinking` / `tool_call` -> 写入 `eval_trace_logs`。
  - 解析到 `annotation` -> 写入 `eval_annotations`。
4. Agent 返回 `done` -> 触发 **Ragas 评分脚本**。
5. 计算结束 -> 更新 `execution_cases` 评分 -> 更新 `eval_tasks.result_summary` -> (Status: `COMPLETED`)。

## 3. 如何实现“面向过程”评估？

通过 `eval_trace_logs` 表。由于你存储了原始的 `event_type`，你可以通过简单的 SQL 统计：

- **工具调用正确率：**统计 `event_type='tool_call'` 且内容符合预期的比例。
- **逻辑严密性：**将 `thinking` 内容发给 LLM-as-a-judge 评分。

## 4. 统计计数 (Counter) 的重要性

在 `executions` 表中，我们设计了 `total_cases`, `completed`, `failed` 等字段。

- **原因：**当数据集很大时（如 1000 条），实时 `COUNT` 数据库会变慢。在后端处理 SSE 时累加这些计数器，前端调用 `status` 接口时能瞬间获得进度百分比。